

Python Lists Reference Guide

Comprehensive 52-section guide covering List Properties, Operations, Built-in Methods, Functions, and Conversions.

#	CATEGORY & TOPIC	BEGINNER-FRIENDLY EXPLANATION	EXAMPLE	OUTPUT / EFFECT
1	PROPERTIES Introduction to List	A list is used to store multiple values in one variable.	<pre>numbers = [10, 20, 30] print(numbers)</pre>	[10, 20, 30]
2	PROPERTIES Creating a List	A list is created using square brackets []. The values inside are called items or elements.	<pre>fruits = ["Apple", "Banana", "Mango"] print(fruits)</pre>	['Apple', 'Banana', 'Mango']
3	PROPERTIES Empty List	An empty list contains no items. We can add items to it later.	<pre>data = [] print(data)</pre>	[]
4	PROPERTIES Mutable	Lists are mutable, which means we can change, add, or remove items after creating them.	<pre>numbers = [10, 20, 30] numbers[0] = 100 print(numbers)</pre>	[100, 20, 30]
5	PROPERTIES Ordered	Lists are ordered, meaning items keep their position and order.	<pre>data = [30, 10, 20] print(data)</pre>	[30, 10, 20]
6	PROPERTIES Duplicates Allowed	A list can contain the same value more than once.	<pre>numbers = [10, 20, 10, 30] print(numbers)</pre>	[10, 20, 10, 30]
7	PROPERTIES Different Data Types	A list can store different types of values together.	<pre>data = [10, "Python", 3.5, True] print(data)</pre>	[10, 'Python', 3.5, True]
8	INDEXING Positive Indexing	Positive indexing starts from 0. The first item is at index 0.	<pre>fruits = ["Apple", "Banana", "Mango"] print(fruits[0])</pre>	Apple
9	INDEXING Negative Indexing	Negative indexing starts from the end. The last item is at index -1.	<pre>fruits = ["Apple", "Banana", "Mango"] print(fruits[-1])</pre>	Mango
10	SLICING List Slicing	Slicing gets a portion of a list using start:stop. The stop position is not included.	<pre>numbers = [10, 20, 30, 40, 50] print(numbers[1:4])</pre>	[20, 30, 40]
11	UPDATING Changing One Item	We can replace an item using its index because lists are mutable.	<pre>numbers = [10, 20, 30] numbers[1] = 200 print(numbers)</pre>	[10, 200, 30]
12	UPDATING Changing Multiple Items	We can use slicing to replace multiple items at once.	<pre>numbers = [10, 20, 30, 40] numbers[1:3] = [200, 300] print(numbers)</pre>	[10, 200, 300, 40]
13	ADDING append()	Adds one item to the end of the list.	<pre>numbers = [1, 2, 3] numbers.append(4) print(numbers)</pre>	[1, 2, 3, 4]
14	ADDING insert()	Adds an item at a specific position.	<pre>numbers = [1, 2, 4] numbers.insert(2, 3) print(numbers)</pre>	[1, 2, 3, 4]
15	ADDING extend()	Adds multiple items from another iterable to the list.	<pre>numbers = [1, 2] numbers.extend([3, 4, 5]) print(numbers)</pre>	[1, 2, 3, 4, 5]

#	CATEGORY & TOPIC	BEGINNER-FRIENDLY EXPLANATION	EXAMPLE	OUTPUT / EFFECT
16	REMOVING remove()	Removes the first matching value from a list.	<pre>numbers = [10, 20, 30, 20] numbers.remove(20) print(numbers)</pre>	[10, 30, 20]
17	REMOVING pop()	Removes and returns an item. Without an index, it removes the last item.	<pre>numbers = [10, 20, 30] numbers.pop() print(numbers)</pre>	[10, 20]
18	REMOVING pop(index)	Removes an item from a specific index.	<pre>numbers = [10, 20, 30] numbers.pop(1) print(numbers)</pre>	[10, 30]
19	REMOVING del	Deletes an item using its index. It can also delete the entire list.	<pre>numbers = [10, 20, 30] del numbers[1] print(numbers)</pre>	[10, 30]
20	REMOVING clear()	Removes all items and leaves an empty list.	<pre>numbers = [1, 2, 3] numbers.clear() print(numbers)</pre>	[]
21	METHODS count()	Counts how many times a specific value appears.	<pre>numbers = [1, 2, 2, 3, 2] print(numbers.count(2))</pre>	3
22	METHODS index()	Returns the position of the first occurrence of a value.	<pre>fruits = ["Apple", "Banana", "Mango"] print(fruits.index("Banana"))</pre>	1
23	METHODS sort()	Sorts the original list in ascending order by default.	<pre>numbers = [30, 10, 20] numbers.sort() print(numbers)</pre>	[10, 20, 30]
24	METHODS sort(reverse=True)	Sorts the original list in descending order.	<pre>numbers = [10, 30, 20] numbers.sort(reverse=True) print(numbers)</pre>	[30, 20, 10]
25	METHODS reverse()	Reverses the current order of the items. It does not sort them.	<pre>numbers = [1, 2, 3, 4] numbers.reverse() print(numbers)</pre>	[4, 3, 2, 1]
26	METHODS copy()	Creates a separate copy of a list.	<pre>a = [1, 2, 3] b = a.copy() b.append(4) print(a) print(b)</pre>	[1, 2, 3] [1, 2, 3, 4]
27	BUILT-IN FUNCTION len()	Returns the total number of items in a list.	<pre>numbers = [10, 20, 30, 40] print(len(numbers))</pre>	4
28	BUILT-IN FUNCTION min()	Returns the smallest value.	<pre>numbers = [10, 5, 20] print(min(numbers))</pre>	5
29	BUILT-IN FUNCTION max()	Returns the largest value.	<pre>numbers = [10, 5, 20] print(max(numbers))</pre>	20
30	BUILT-IN FUNCTION sum()	Adds all numeric values in a list.	<pre>numbers = [10, 20, 30] print(sum(numbers))</pre>	60
31	BUILT-IN FUNCTION sorted()	Returns a new sorted list without changing the original list.	<pre>numbers = [30, 10, 20] print(sorted(numbers))</pre>	[10, 20, 30]
32	BUILT-IN FUNCTION any()	Returns True if at least one item is considered True.	<pre>data = [0, 0, 5, 0] print(any(data))</pre>	True

#	CATEGORY & TOPIC	BEGINNER-FRIENDLY EXPLANATION	EXAMPLE	OUTPUT / EFFECT
33	BUILT-IN FUNCTION all()	Returns True only if all items are considered True.	<pre>data = [1, 5, 10] print(all(data))</pre>	True
34	BUILT-IN FUNCTION list()	Converts an iterable into a list.	<pre>text = "ABC" print(list(text))</pre>	['A', 'B', 'C']
35	BUILT-IN FUNCTION reversed()	Returns items in reverse order. We use list() to display them as a list.	<pre>numbers = [1, 2, 3] print(list(reversed(numbers)))</pre>	[3, 2, 1]
36	BUILT-IN FUNCTION enumerate()	Gives both the index and value while working with list items.	<pre>fruits = ["Apple", "Mango"] print(list(enumerate(fruits)))</pre>	[(0, 'Apple'), (1, 'Mango')]
37	BUILT-IN FUNCTION zip()	Combines corresponding items from two or more lists.	<pre>names = ["A", "B"] marks = [80, 90] print(list(zip(names, marks)))</pre>	[('A', 80), ('B', 90)]
38	OPERATORS Concatenation +	Joins two lists together.	<pre>a = [1, 2] b = [3, 4] print(a + b)</pre>	[1, 2, 3, 4]
39	OPERATORS Repetition *	Repeats the items of a list.	<pre>numbers = [1, 2] print(numbers * 3)</pre>	[1, 2, 1, 2, 1, 2]
40	OPERATORS Membership in	Checks whether an item exists in a list.	<pre>numbers = [10, 20, 30] print(20 in numbers)</pre>	True
41	OPERATORS Membership not in	Checks whether an item does not exist in a list.	<pre>numbers = [10, 20, 30] print(50 not in numbers)</pre>	True
42	CONCEPT Looping Through a List	A for loop allows us to access every item one by one.	<pre>fruits = ["Apple", "Banana"] for fruit in fruits: print(fruit)</pre>	Apple Banana
43	CONCEPT Nested Lists	A nested list is a list containing other lists.	<pre>matrix = [[1, 2], [3, 4]] print(matrix[0][1])</pre>	2
44	CONCEPT List Aliasing	Two variables refer to the same list, so changes through one affect the other.	<pre>a = [1, 2, 3] b = a b.append(4) print(a)</pre>	[1, 2, 3, 4]
45	CONCEPT Copying vs Aliasing	A copy creates a separate list, while aliasing creates another name for the same list.	<pre>a = [1, 2] b = a.copy() b.append(3) print(a)</pre>	[1, 2]
46	CONCEPT List Comprehension	A shorter way to create a new list using a loop.	<pre>numbers = [1, 2, 3, 4] squares = [x * x for x in numbers] print(squares)</pre>	[1, 4, 9, 16]
47	CONVERSION String to List	We can convert a string into a list of characters using list().	<pre>text = "Python" result = list(text) print(result)</pre>	['P', 'y', 't', 'h', 'o', 'n']
48	CONVERSION Tuple to List	We can convert a tuple into a list using list().	<pre>data = (10, 20, 30) result = list(data) print(result)</pre>	[10, 20, 30]
49	CONVERSION Set to List	We can convert a set into a list using list(). The order may vary because sets are unordered.	<pre>data = {10, 20, 30} result = list(data) print(result)</pre>	[10, 20, 30] (order may vary)

#	CATEGORY & TOPIC	BEGINNER-FRIENDLY EXPLANATION	EXAMPLE	OUTPUT / EFFECT
50	CONVERSION Range to List	list() can convert numbers generated by range() into a list.	<pre>result = list(range(1, 6)) print(result)</pre>	[1, 2, 3, 4, 5]
51	CONVERSION List to Tuple	We can convert a list into a tuple using tuple().	<pre>data = [10, 20, 30] result = tuple(data) print(result)</pre>	(10, 20, 30)
52	CONVERSION List to Set	We can convert a list into a set using set(). Duplicate values are automatically removed.	<pre>data = [1, 2, 2, 3, 3] result = set(data) print(result)</pre>	{1, 2, 3}